

Metadata-Driven ETL Framework using Databricks, PySpark, Delta Lake, and YAML

Project Overview

This project implements a **Metadata-Driven ETL Framework** in **Databricks** using **PySpark**, **Delta Lake**, **Unity Catalog**, and **YAML configuration files**.

The framework follows the **Medallion Architecture (Bronze → Silver → Gold)** where each layer reads metadata from YAML files, processes the data, and generates metadata for the next layer. This design minimizes hard-coded logic and enables onboarding of new datasets by updating only YAML configuration files instead of modifying the pipeline code.

Objectives

The primary objectives of this project are:

- Develop a reusable ETL framework.
 - Eliminate hard-coded table names and file paths.
 - Use YAML files for configuration and transformation rules.
 - Automatically generate metadata after each processing layer.
 - Build Bronze, Silver, and Gold layers following Medallion Architecture.
 - Create a scalable and metadata-driven ETL solution.
-

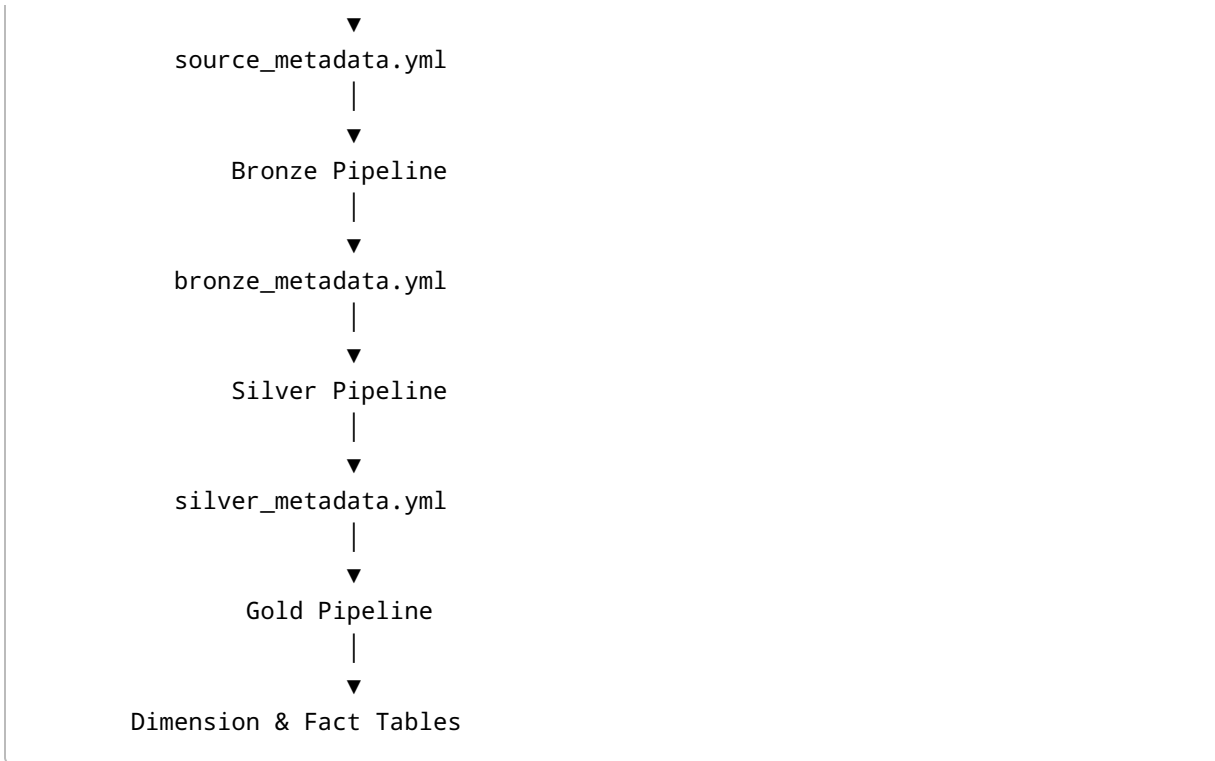
Technology Stack

- Databricks
 - Apache Spark (PySpark)
 - Delta Lake
 - Unity Catalog
 - YAML
 - Python
-

Project Architecture

Source Data





Catalog Structure

The project is organized inside the Unity Catalog **ETL_FrameWork_Metadata**.

```
ETL_FrameWork_Metadata
|
├─ bronze
├─ silver
├─ gold
├─ config
├─ generated
├─ source_data
└─ default
```

Schema Description

Schema	Purpose
source_data	Stores the raw CSV source files
config	Stores all YAML configuration files

Schema	Purpose
bronze	Raw Delta tables
silver	Cleaned and transformed Delta tables
gold	Dimension and Fact tables
generated	Stores metadata YAML files generated by the pipelines

Folder Structure

```
ETL_Framework/  
|  
├─ config/  
│   ├── bronze01.yml  
│   ├── silver_rules.yml  
│   └── gold_rules.yml  
|  
├─ generated/  
│   ├── bronze_metadata.yml  
│   ├── silver_metadata.yml  
│   └── gold_metadata.yml  
|  
├─ source_data/  
│   ├── customers.csv  
│   ├── products.csv  
│   └── sales.csv  
|  
├─ Bronze Pipeline.py  
├─ Silver Pipeline.py  
├─ Gold Pipeline.py  
└─ README.md
```

Step 1 : Environment Setup

The first step is creating the Unity Catalog and required schemas.

Created Catalog:

```
ETL_FrameWork_Metadata
```

Created Schemas:

- config
- bronze
- silver
- gold
- generated
- source_data

These schemas organize configuration files, metadata, source files, and processed Delta tables.

Step 2 : Source Metadata Configuration

The Bronze layer starts by reading the source metadata YAML file.

Example:

```
bronze01.yml
```

This file contains:

- Source data location
- Dataset names
- Source file names
- Target Bronze table names

Example configuration:

```
customers.csv  
products.csv  
sales.csv
```

This allows onboarding a new dataset by simply adding another entry into the YAML file.

No code changes are required.

Step 3 : Bronze Layer

Purpose

The Bronze layer ingests raw source data into Delta tables without applying business transformations.

Processing Steps

1. Read the source YAML.
2. Loop through every dataset.
3. Read the CSV file.
4. Infer schema automatically.
5. Create Delta tables in the Bronze schema.
6. Save tables.
7. Generate Bronze metadata.

Bronze Tables Created

- customers
- products
- sales

Stored as:

```
ETL_FrameWork_Metadata.bronze.customers  
ETL_FrameWork_Metadata.bronze.products  
ETL_FrameWork_Metadata.bronze.sales
```

Bronze Metadata Generation

After loading the Bronze tables, the pipeline automatically generates:

```
bronze_metadata.yml
```

Example:

```
catalog: ETL_FrameWork_Metadata  
schema: bronze  
  
datasets:
```

- table: customers
path: ETL_FrameWork_Metadata.bronze.customers
- table: products
path: ETL_FrameWork_Metadata.bronze.products
- table: sales
path: ETL_FrameWork_Metadata.bronze.sales

The Bronze metadata contains only Bronze table information, making it independent of the source files.

Step 4 : Silver Layer

Purpose

The Silver layer cleans and standardizes the Bronze data using metadata-driven transformation rules.

The Silver pipeline reads:

- bronze_metadata.yml
- silver_rules.yml

No Bronze table names are hard-coded.

Silver Transformations

The framework supports the following transformations:

Common Transformations

- Trim spaces
- Remove duplicate records
- Remove null values

Column Transformations

- Rename columns
- Cast data types
- Convert to uppercase
- Convert to lowercase
- Drop unwanted columns
- Add calculated columns

Example:

```
Rename  
  
customerid  
↓  
customer_id  
  
Cast  
  
price  
↓  
Double  
  
Uppercase  
  
category  
  
Lowercase  
  
email  
  
Add Column  
  
load_date = current_date()
```

Silver Tables Created

```
customers  
  
products  
  
sales
```

Stored in:

```
ETL_FrameWork_Metadata.silver
```

Silver Metadata Generation

After processing, the Silver pipeline generates:

```
silver_metadata.yml
```

Example:

```
catalog: ETL_Framework_Metadata
schema: silver

datasets:

- table: customers
- table: products
- table: sales
```

This metadata becomes the input for the Gold layer.

Step 5 : Gold Layer

Purpose

The Gold layer creates analytical datasets.

It reads:

- silver_metadata.yml
- gold_rules.yml

The Gold layer dynamically performs joins based on metadata.

Gold Processing

The pipeline performs the following:

1. Read Silver tables.
2. Create Dimension tables.
3. Join datasets.

4. Build Fact table.
5. Save Gold tables.
6. Generate Gold metadata.

Dimension Tables

Generated automatically:

```
dim_customers  
  
dim_products
```

Fact Table

Generated:

```
fact_sales
```

The joins are entirely controlled by the YAML configuration.

Example:

```
Sales  
|  
├── Customer  
└── Product
```

No join conditions are hard-coded.

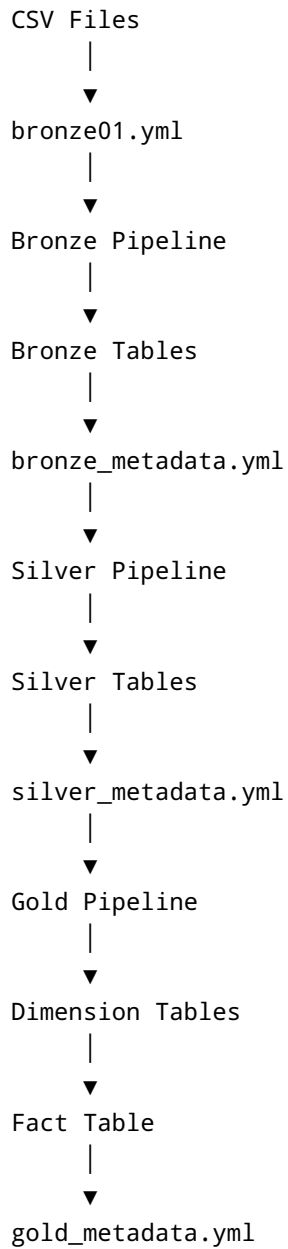
Gold Metadata Generation

The Gold pipeline generates:

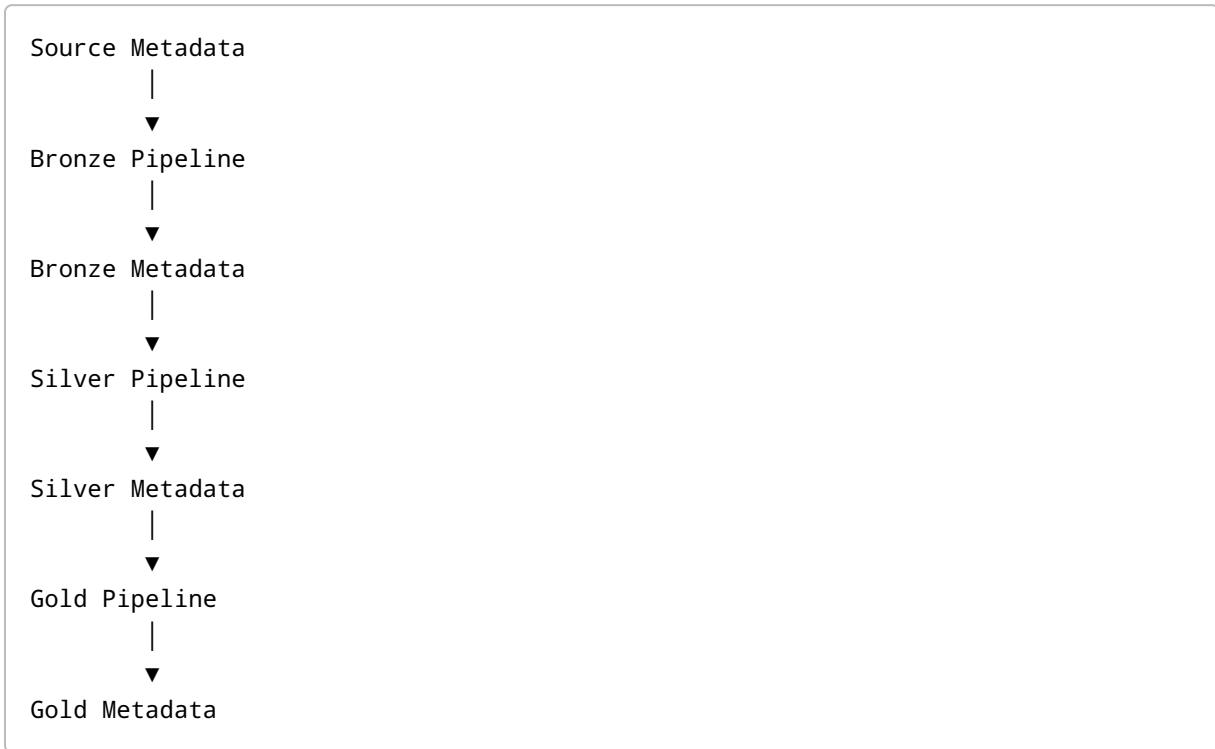
```
gold_metadata.yml
```

which contains the locations of all Gold tables.

End-to-End Data Flow



Metadata Flow



Each pipeline consumes the metadata generated by the previous layer, creating a fully metadata-driven workflow.

Advantages of the Framework

- Fully metadata-driven architecture.
 - No hard-coded dataset names.
 - Easy onboarding of new datasets.
 - Reusable ETL pipelines.
 - Clear separation of configuration and processing logic.
 - Scalable and maintainable design.
 - Supports Medallion Architecture.
 - Automatic metadata generation after every layer.
 - Dynamic transformation rules through YAML.
 - Easily extendable for additional transformation types and datasets.
-

Future Enhancements

The framework can be enhanced by adding:

- Surrogate key generation for dimension tables.
 - Slowly Changing Dimensions (SCD Type 1 and Type 2).
 - Data quality validation framework.
 - Incremental and CDC loading.
 - Logging and audit tables.
 - Error handling and retry mechanisms.
 - Dynamic schema evolution.
 - Workflow orchestration using Databricks Workflows or Apache Airflow.
 - Automated unit testing.
 - Data lineage and monitoring.
-

Conclusion

This project successfully demonstrates a complete metadata-driven ETL framework using Databricks. By separating configuration from processing logic and generating metadata at each stage, the framework provides a scalable, reusable, and maintainable solution for modern data engineering workflows. The layered Medallion Architecture, combined with YAML-driven configurations, allows new datasets to be integrated with minimal effort while ensuring consistency, flexibility, and ease of maintenance.